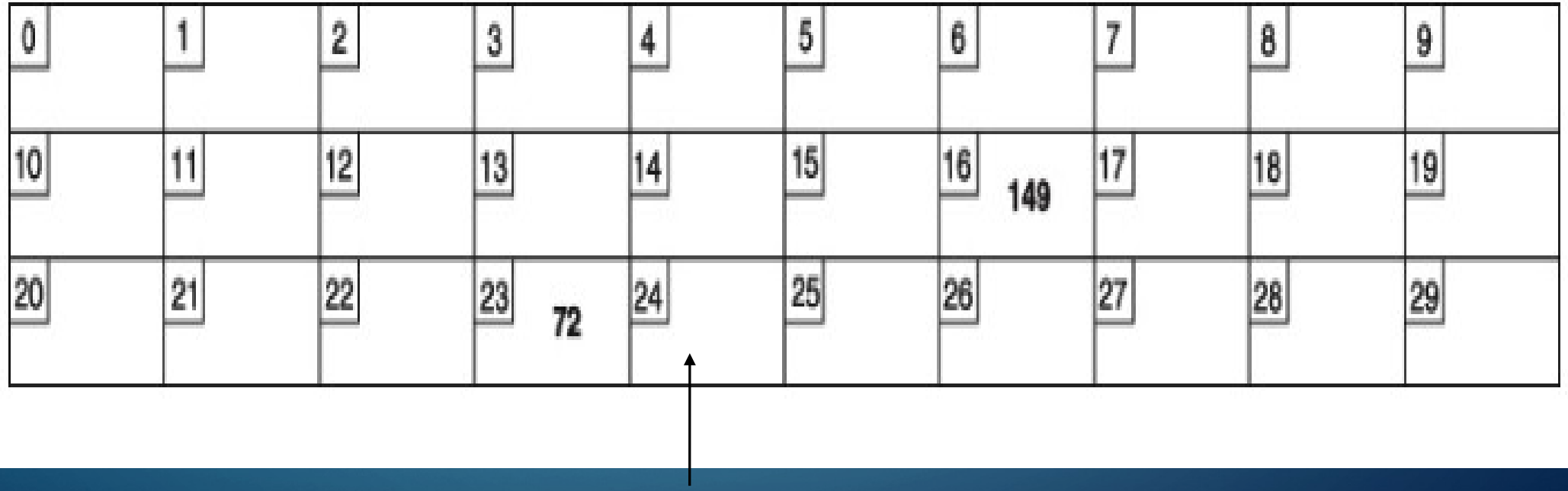
DATA REPRESENTATION IN COMPUTER SYSTEMS

COMPUTER MEMORY (RAM)



BYTE

BIT ?!

- **The most basic unit of information in a digital computer is called a bit, which is a contraction of binary digit. In the concrete sense, a bit is nothing more than a state of “on” or “off ” (or “high” and “low”) within a computer circuit**

BIT VS BYTE

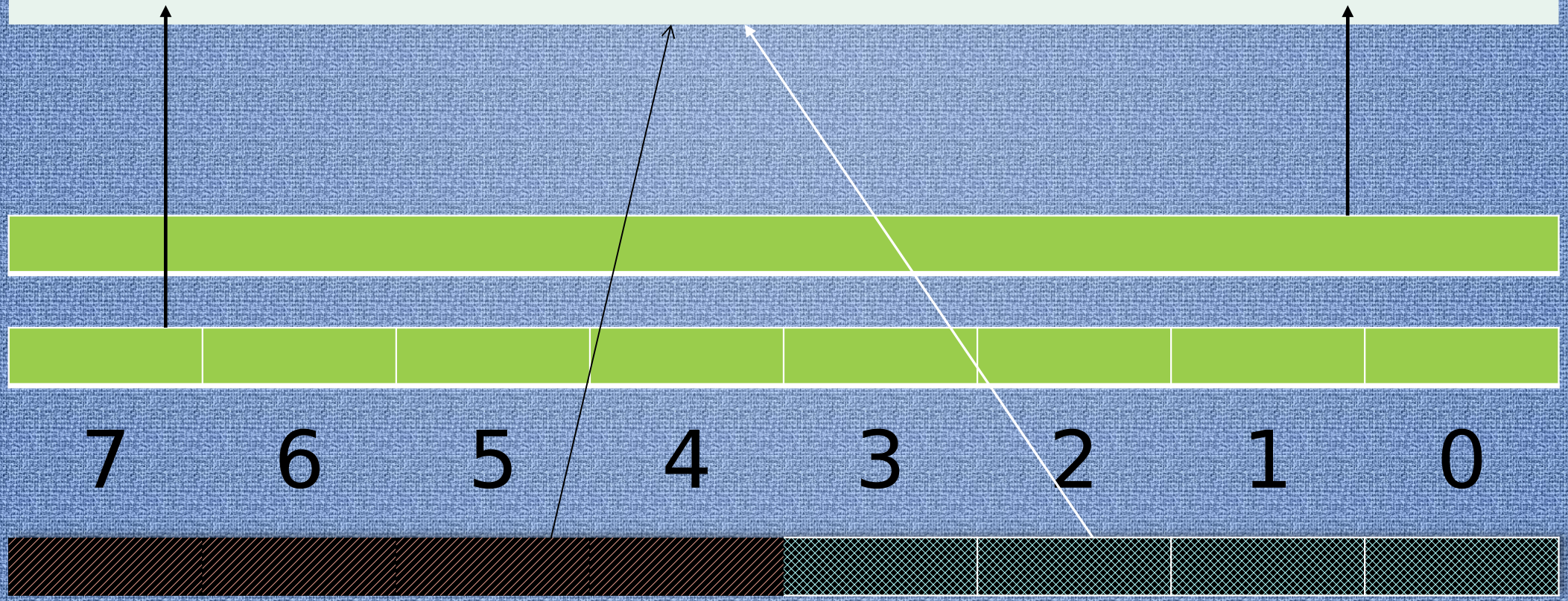
- A **bit** is the most basic unit of information in a computer.
- It is a state of “on” or “off” in a digital circuit.
- Sometimes these states are “high” or “low” voltage instead of “on” or “off.”
- A **byte** is a group of eight bits.
- A byte is the smallest possible addressable unit of computer storage.
- The term, “**addressable**,” means that a particular byte can be retrieved according to its location in memory.

BIT VS BYTE

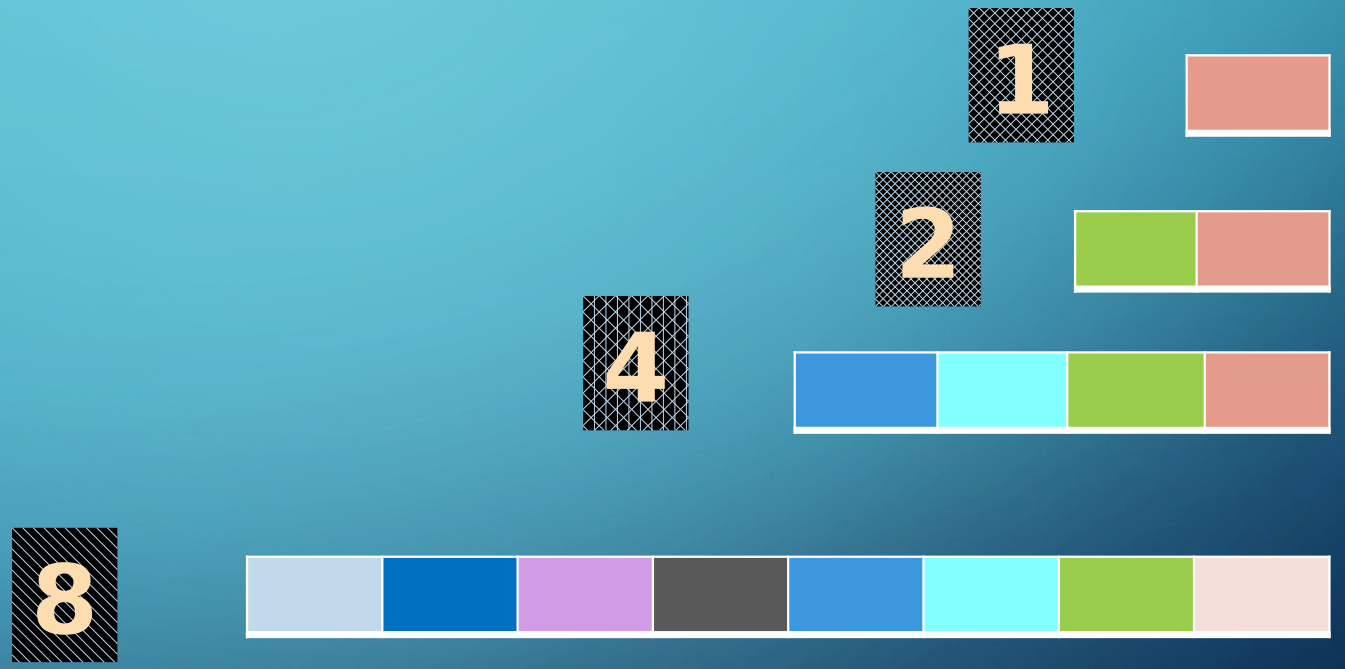
NIBBLE ?!

- **An 8-bit byte can be divided into two 4-bit halves called nibbles (or nybbles)**
- **Because each bit of a byte has a value within a positional numbering system, the nibble containing the least-valued binary digit is called the low-order nibble, and the other half the high-order nibble.**

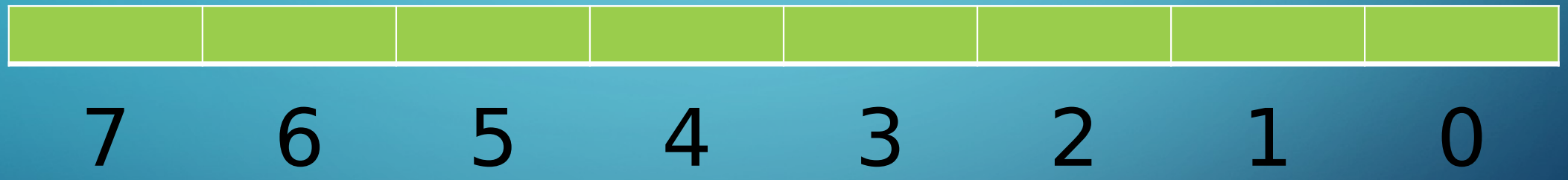
BIT VS NIBBLE VS BYTE



WORD SIZE ?



SIGNED INTEGER REPRESENTATION (WORD : 8 BITS (1 BYTE))

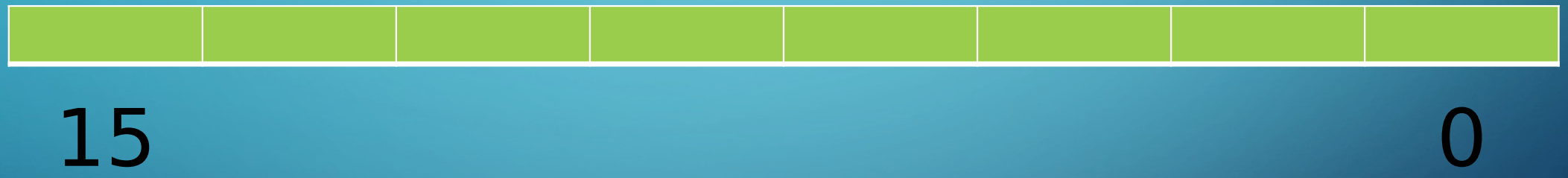


SIGNED INTEGER REPRESENTATION (WORD : 8 BITS)

7 6 5 4 3 2 1 0

Sign Bit

UNSIGNED INTEGER REPRESENTATION (WORD : 16 BITS (2 BYTES))

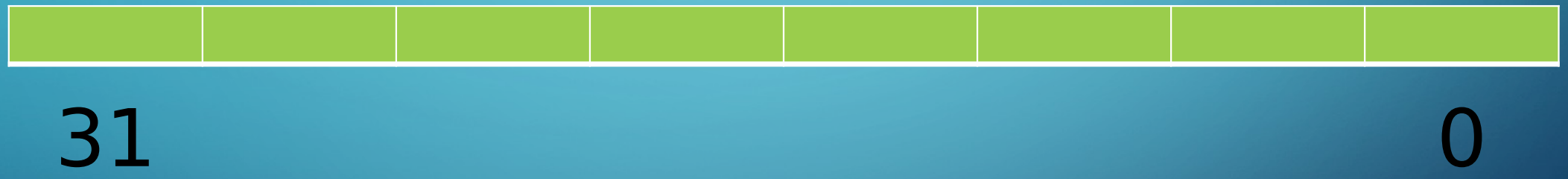


SIGNED INTEGER REPRESENTATION (WORD : 16 BITS (2 BYTES))

15 14 1 0

Sign Bit

UNSIGNED INTEGER REPRESENTATION (WORD : 32 BITS (4 BYTES))



SIGNED INTEGER REPRESENTATION (WORD : 32 BITS (4 BYTES))

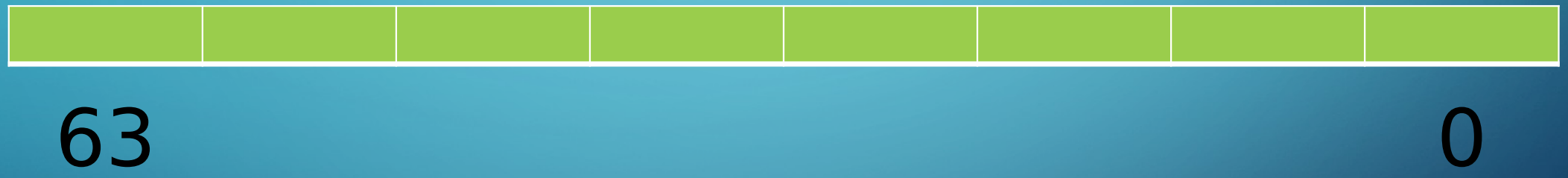
31 30 1 0

Sign Bit

OBJECTIVES:

- Numeral systems in computer systems
- Hexadecimal number System
- Octal Number System
- Floating-point number representation in Computer

UNSIGNED INTEGER REPRESENTATION (WORD : 64 BITS (8 BYTES))



SIGNED INTEGER REPRESENTATION (WORD : 64 BITS (8 BYTES))

64 63 1 0

Sign Bit

UNSIGNED INTEGER VALUE RANGE

$0 \ldots 2^n - 1$	$0 \ldots 2^n - 1$	$0 \ldots 2^n - 1$
$0 \ldots 2^8 - 1$	$0 \ldots 2^{16} - 1$	$0 \ldots 2^{32} - 1$
$0 \ldots 255$	$0 \ldots 65535$	$0 \ldots$

UNSIGNED INTEGER VALUE RANGE

$0 \ldots 2^n - 1$	$0 \ldots 2^n - 1$	$0 \ldots 2^n - 1$
$0 \ldots 2^8 - 1$	$0 \ldots 2^{16} - 1$	$0 \ldots 2^{32} - 1$
$0 \ldots 255$	$0 \ldots 65535$	$0 \ldots$

Size/Type	Range
1 byte unsigned	0 to 255
2 byte unsigned	0 to 65,535
4 byte unsigned	0 to 4,294,967,295
8 byte unsigned	0 to 18,446,744,073,709,551,615

```
unsigned short us;
unsigned int ui;
unsigned long ul;
unsigned long long ull;
```

SIGNED INTEGER VALUE RANGE

1 BYTE	$-2^{n-1} \dots 2^{n-1}-1$	$-2^{8-1} \dots 2^{n-1}-1$	$-2^{7} \dots 2^{7}-1$	-128 ... 127
2 BYTES	$-2^{n-1} \dots 2^{n-1}-1$	$-2^{16-1} \dots 2^{16-1}-1$	$-2^{15} \dots 2^{15}-1$	-32768 ... 32767
3 BYTES	$-2^{n-1} \dots 2^{n-1}-1$	$-2^{32-1} \dots 2^{32-1}-1$	$-2^{31} \dots 2^{31}-1$	-21474833648 ... 21474833647

```
short s;
int i;
long l;
long long ll;
```

DATA TYPES IN C++

Table 2-6 Integer Data Types

Data Type	Typical Size	Typical Range
`short int`	2 bytes	–32,768 to +32,767
`unsigned short int`	2 bytes	0 to +65,535
`int`	4 bytes	–2,147,483,648 to +2,147,483,647
`unsigned int`	4 bytes	0 to 4,294,967,295
`long int`	4 bytes	–2,147,483,648 to +2,147,483,647
`unsigned long int`	4 bytes	0 to 4,294,967,295
`long long int`	8 bytes	–9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
`unsigned long long int`	8 bytes	0 to 18,446,744,073,709,551,615

DATA TYPES IN JAVA

Table 2-5 Primitive Data Types for Numeric Data

Data Type	Size	Range
`byte`	1 byte	Integers in the range of -128 to $+127$
`short`	2 bytes	Integers in the range of $-32,768$ to $+32,767$
`int`	4 bytes	Integers in the range of $-2,147,483,648$ to $+2,147,483,647$
`long`	8 bytes	Integers in the range of $-9,223,372,036,854,775,808$ to $+9,223,372,036,854,775,807$
`float`	4 bytes	Floating-point numbers in the range of $\pm 3.4 \times 10^{-38}$ to $\pm 3.4 \times 10^{38}$, with 7 digits of accuracy
`double`	8 bytes	Floating-point numbers in the range of $\pm 1.7 \times 10^{-308}$ to $\pm 1.7 \times 10^{308}$, with 15 digits of accuracy

HEXADECIMAL NUMERAL SYSTEM

- Binary numbers are often expressed in **hexadecimal** to improve their readability
- The hexadecimal numeral system, often shortened to "**hex** ”
- Hexadecimal numerals are widely used by computer system designers and programmers because they **provide a human-friendly representation of binary-coded values**
- Hexadecimal numbers are used to represent

HEXADECIMAL NUMERAL SYSTEM

(0…9,A,B,C,D,E,F)16

DEC	HEX	BINARY
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

COLOR PICKER

HEXADECIMAL NUMERAL SYSTEM

	BYTE								
HEXADECIMAL	40	32	24	16	8	4	2	1	DECIMAL
	8	4	2	1	8	4	2	1	
12	0	0	0	1	0	0	1	0	18
37	0	0	1	1	0	1	1	1	55
2B	0	0	1	0	1	0	1	1	43
A3	1	0	1	0	0	0	1	1	163
4C	0	1	0	0	1	1	0	0	76

OCTAL NUMERAL SYSTEM

(0,1,2,3,4,5,6,7)8

DEC	OCT	BINARY
0	0	000
1	1	001
2	2	010
3	3	011
4	4	100
5	5	101
6	6	110
7	7	111

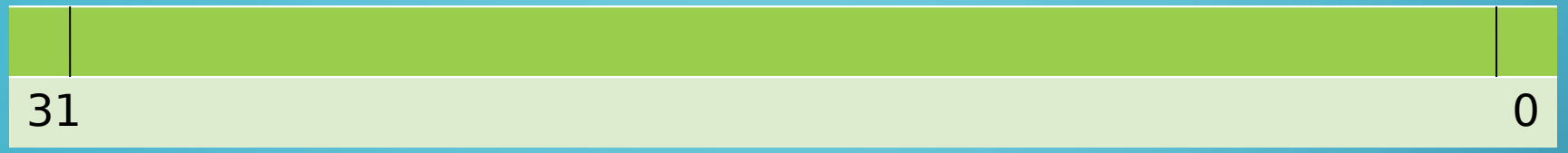
OCTAL NUMERAL SYSTEM

	BYTE								
OCTAL	128	64	32	16	8	4	2	1	DECIMAL
	2	1	4	2	1	4	2	1	
012	0	0	0	0	1	0	1	0	10
367	1	1	1	1	0	1	1	1	247
123	0	1	0	1	0	0	0	1	81
345	1	1	1	0	0	1	0	1	229
246	1	0	1	0	0	1	1	0	166

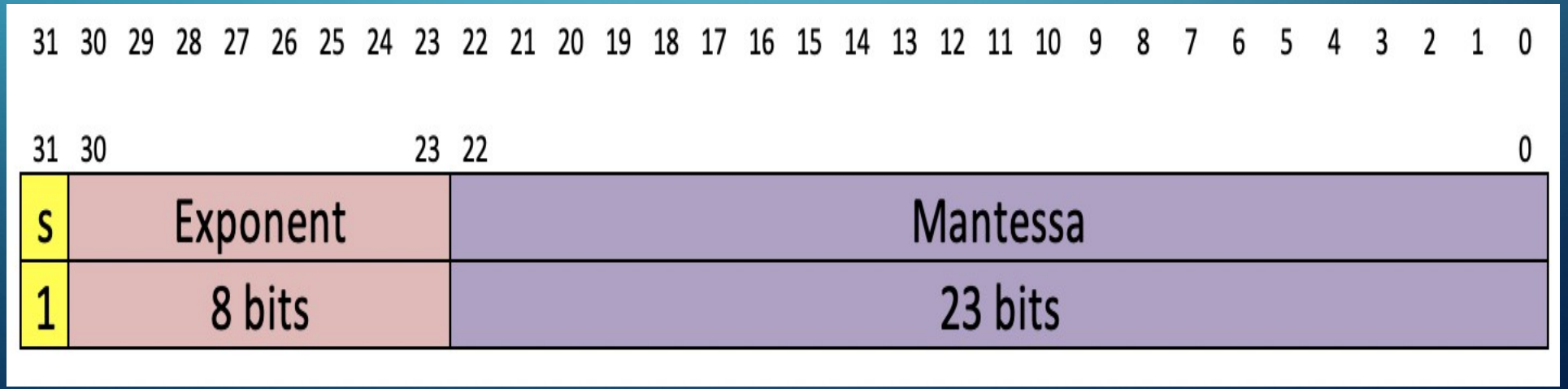
NUMERAL SYSTEMS IN COMPUTER SYSTEMS

(0 … 1)2	**(0,1)2**	**BINARY**
(0 … 9)10	(0,1,2,3,4,5,6,7,8,9)10	DECIMAL
(0 … 7)8	(0,1,2,3,4,5,6,7)8	OCTAL
(0 … 2)16	(0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F)16	HEXADECIMAL

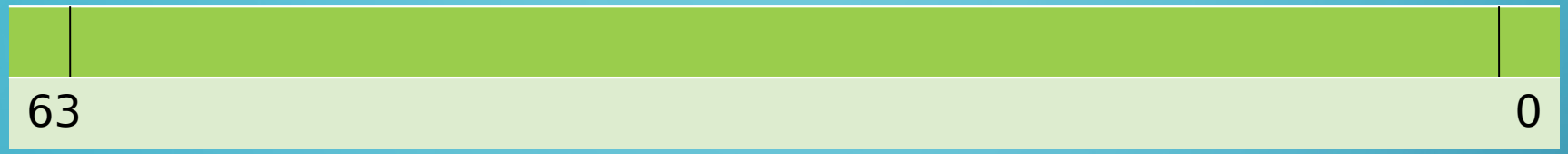
FLOATING-POINT NUMERAL SYSTEMS
SINGLE PRECISION (4 BYTES)
IEEE–754



4 BYTES - 32 BITS



FLOATING-POINT NUMERAL SYSTEMS
DOUBLE PRECISION (8 BYTES)
IEEE–754



8 BYTES - 64 BITS

64 BITS		
63	62 52	51 0
s	EXPONENT	MANTESSA
1	11	52

EXPONENTIAL NOTATION

- Exponential notation is a method of simplifying the writing and handling of very large or very small numbers
- To express a number in exponential notation, write it in the form: **C × 10 ^ N**

  where **C** is a number between **1** and **9**

Number	EXPONENTIAL NOTATION	
60000	6 * 10 ^ 4	-> 6E4
2000	2 * 10 ^ 3	-> 2E3
800	8 * 10 ^ 2	-> 8E2
2230	2.30 * 10 ^ 3	-> 2.3E3
672	6.72 * 10 ^ 2	-> 6.72E2
324561	3.24561 * 10 ^ 5	-> 3.24561E5
87659	8.7659 * 10 ^ 4	->

EXPONENTIAL NOTATION

Number	EXPONENTIAL NOTATION
0.00056	5.6 * 10 ^ 4
2.00016	2.16 * 10 ^ -3
324561	3.24561 * 10 ^ 5
87659	8.7659 * 10 ^ 4

HOW TO CONVERT A FLOATING-POINT NUMBER TO BINARY (SINGLE PRECISION)

1. CONVERT THE INTEGER NUMBER TO BINARY
2. CONVERT THE DECIMAL NUMBERS TO BINARY
3. CONVERT YOUR NUMBERS TO EXPONENTIAL NOTATION FORMAT
4. ADD BIAS TO 127 AND STORE THE RESULT IN EXPONENT SECTION OF 32 BITS
5. PUT 1 IN SIGN BIT IF NUMBER IS NEGATIVE
6. STORE THE BINARY DIGITS IN MANTESSA SECTION OF 4 BYTES

HOW TO CONVERT A FLOATING-POINT NUMBER TO BINARY (DOUBLE PRECISION)

1. CONVERT THE INTEGER NUMBER TO BINARY
2. CONVERT THE DECIMAL NUMBERS TO BINARY
3. CONVERT YOUR NUMBERS TO EXPONENTIAL NOTATION FORMAT
4. ADD BIAS TO 1023 AND STORE IN **EXPONENT SECTION** OF 8 BITS
5. PUT 1 IN SIGN BIT IF NUMBER IS NEGATIVE
6. STORE THE BINARY DIGITS IN **MANTESSA SECTION** OF 4 BYTES